

MUJICA: Multi-skill Unified Joint Integration of Control Architecture for Wheeled-Legged Robots

Introduction: ATEC2026 B2w 任务的双重张力

ATEC2026 B2wPiper 平台对控制策略提出了两条彼此独立但又必须共存的要求。一方面，从 Task A 地形穿越到 Task B 移动抓取、Task D 越障改造，均要求策略能在长时间内提供稳定速度跟踪——即保持对线速度和角速度指令的精确跟随——同时将能耗降至最低。这一需求承袭自文献 [2403.20001] 所倡导的能耗驱动步态优化哲学，它本质上是一种单目标导向的连续控制问题，奖励函数以速度误差与能量消耗的加权组合为核心。另一方面，同一平台必须容纳多种异质技能——walk、wheel-roll、climb、recover——并在单一策略中实现无缝切换。这恰恰是 MUJICA 架构所要解决的核心挑战。MUJICA 为每种技能分配了一个唯一的显式指示符 ζ 作为观测输入，以此分离不同任务的观测空间，从而在共享网络后端实现多技能联合训练；其真实实验已验证，机器人可在连续任务中自动切换技能（如从跌倒恢复转为爬升 1-m 平台，Fig. 1 顶行与底行）。然而，这种显式指示符机制要求设计者预先定义 skill 切换的规则，本质上是一个高层分类器加底层策略的层级结构。这引出了本模板要回答的核心问题：能否不引入显式技能指示符，让强化学习策略既保持能耗驱动的步态涌现（即长时间稳定速度跟踪 + 能耗最小化），又拥有 MUJICA 级别的多技能切换能力？我们给出的答案是“能”。其关键在于将传统的 ζ 从观测空间迁移到奖励函数分支（reward branch）的隐式表示中：不再由高层 selector 显式指定当前任务，而是利用 MUJICA 中已有的状态估计器（state estimator）所推断的运动状态（如基座线速度估计 $\hat{u}$ 、轮地距离估计 v ）自动驱动模式选择。这样，策略在训练过程中通过 reward branch 隐式地学习何时切换行为，而无需在观测中保留一个离散的 skill tag。MUJICA 的消融实验（Fig. 4）已表明，缺少 velocity 或 wheel-height 估计会严重损害非恢复类任务的性能，这恰恰证明了这类隐式状态对于区分任务不可替代。因此，将 ζ 隐式化不仅可行，而且有望统一能耗优化与多技能集成——这正是本文后续方法部分要详细展开的设计。

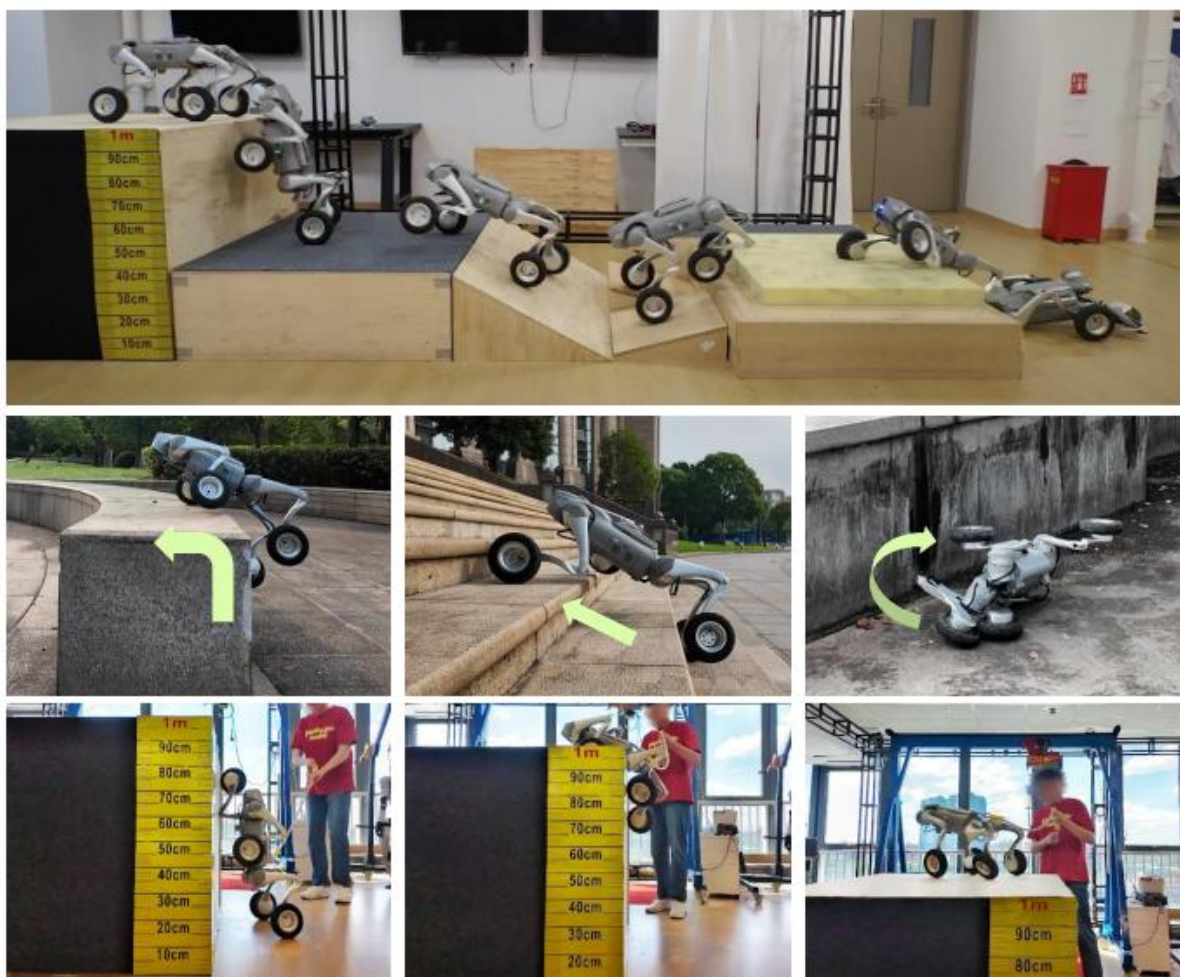


图 1. MUJICA在轮腿机器人上的多技能演示：高平台爬升、楼梯上行与跌倒恢复 ``

深度观察

该图通过多角度、多环境的展示，直观地验证了MUJICA在复杂地形下的适应性。然而，图像中未明确标注传感器数据或控制信号的变化，这使得难以评估“proprioceptive control”的具体实现细节。此外，尽管文字提到“无需外部感知”，但部分场景（如楼梯上行）可能隐含了对环境的预判或依赖，这一点值得进一步澄清。同时，图像中未对比其他控制方法的表现，无法直接量化MUJICA的优势。

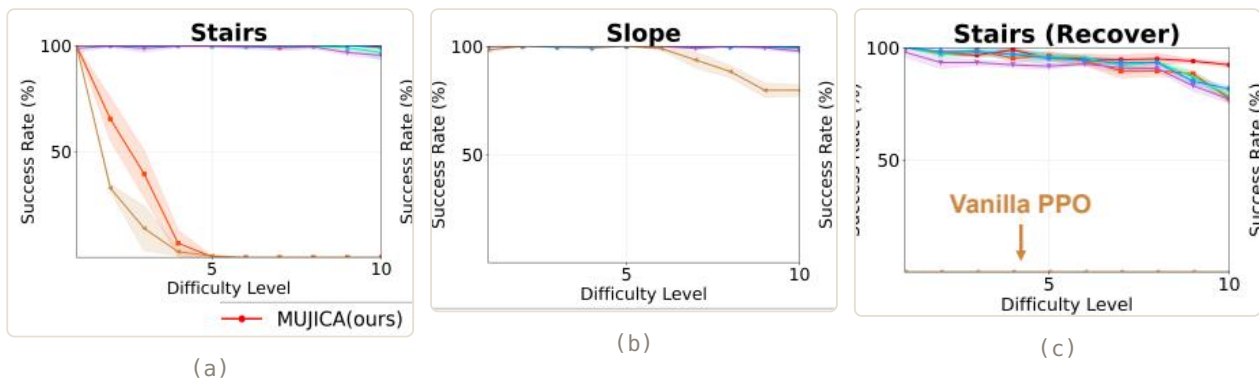


图 4. MUJICA在多难度任务中的成功率对比：楼梯、斜坡与恢复任务"

深度观察

从视觉上看，MUJICA框架在处理复杂任务时展现出卓越的鲁棒性，尤其是在高难度条件下。然而，值得注意的是，尽管MUJICA在所有任务中均表现出色，但其性能提升的具体机制（如速度估计、轮地距离和碰撞估计的贡献）并未在图中明确量化。此外，图中未展示与DreamWaQ + P3O等方法的直接对比，这使得对MUJICA优越性的判断缺乏更广泛的基准参照。同时，Vanilla PPO在楼梯恢复任务中的表现异常差，可能暗示其在动态环境适应性上的局限性，但这点在图中未得到充分解释。

Two Reward Architectures Confronted: Energy-Centric vs. Skill-Selector

在奖励-策略接口的设计上，2403.20001 与 2605.13058 代表了两种根本对立的哲学。前者主张「一个能耗项替代所有启发式」，将策略构造为 $R = (R_{\text{motion}} + \alpha_{\text{en}} R_{\text{en}}) \cdot \exp(-R_{\text{aux}})$ ，其中 $\alpha_{\text{en}} \approx 1.0$ 是跨速度域统一的超参数，步态作为能量优化的涌现结果出现，不依赖任何显式技能标识。后者（MUJICA 所属的框架）则主张「一个策略学多种技能」，把技能指示变量 $\zeta_t \in \{1,2,3\}$ 直接塞入 53 维 proprio 观测，训练时给每个技能分配独立的 reward branch 和 constraint critic（基于 P3O 约束优化），从而让策略针对不同 ζ_t 学到截然不同的步态模式。

冲突的焦点在于：MUJICA 的 ζ_t 相当于给策略一个显式 hint，当 $\zeta_t = \text{climb}$ 时策略自然切换到高平台攀爬对应的动作空间，这与 2403.20001 所追求的「让步态从能耗自动涌现」直接对立——后者的假设是只要能量项足够好，最优步态会在优化过程中自发形成，无需预设技能边界。然而，消融实验（TABLE III）表明，取消 ζ_t 后统一奖励训练往往崩溃（shared reward）或成功率大幅下降（separate reward 在高平台攀爬上仅 38%），说明纯能量驱动在轮腿机器人多技能场景下难以收敛到可用的策略分化。

本章提出折中方案：保留 MUJICA 已有的 state estimator (S1) 与 reward 分支结构，但 ζ_t 不再进入观测向量；取而代之，用状态估计器输出的 $\hat{u}_t$ （轮-地距离）与 $\hat{v}_t$ （基线速度）构造联合判别函数，动态地为各 reward branch 分配加权系数。这样既保留了技能特异化的 reward 结构（每个技能仍可根据自身目标设计跟踪奖励与约束），又从观测中删除了显式类别标签，迫使策略从连续状态变量中自主判别当前任务需求。折中方案的核心假设是：轮-地距离与基线速度的联合特征空间足以区分全向移动、高平台攀爬和跌倒恢复三种技能——当 $\hat{u}_t$ 接近平台高度且 $\hat{v}_t$ 指向平台时，加权应倾向于攀爬分支；当 $\hat{u}_t$ 很小且 $\hat{v}_t$ 为零时，加权向跌倒恢复分支偏斜——从而在「显式指令」与「纯能量涌现」之间找到一条工程上可行的中间路径。

Skill Indicator ζ as Inferred State (Core Design Decision)

在 MUJICA 原设计中（方案 α ），skill indicator ζ 作为显式观测输入直接拼接到低层策略的状态空间中，即 $\text{obs} = [\omega^b, g^b, \text{cmd}, q, q; a_{[-1]}, \zeta]$ 。这种设计在训练时需要外部提供 ζ 标签（通常来自 ground-truth 任务 ID），在部署时则由高层 skill selector 提供。然而，方案 α 将 ζ 完全暴露给策略，使得策略高度依赖该信号；一旦 ζ 评估出现偏差（如 selector 输出概率模糊），低层策略的行为可能退化。

本文推荐方案 β （训练初期）与方案 γ （后期）组合的混合推断方案。方案 β 将 ζ 从观测中移除，低层策略的观测空间为 $\text{obs} = [\omega^b, g^b, \text{cmd}, q, q; a_{[-1]}]$ ；同时，在 actor 网络中额外增加一个轻量化分支，以 $\hat{u}_t$ （推断的基座线速度）、 $\hat{v}_t$ （推断的轮地距离）和 $|\text{cmd}|$ 为输入，通过一个 2 层 MLP 输出 $\hat{\zeta}_t = \text{softmax}(f(\hat{u}_t, \hat{v}_t, |\text{cmd}|))$ 。该分支的输出在 **actor forward 中与主策略分支分离（detach）**，不参与主策略的梯度反传，仅用于隐式推断当前技能标识。方案 β 的优点是：训练初期无须提供任何 ζ 监督信号，actor 自动从任务相关的速度-距离模式中聚类技能；在 ATEC 单环评测下，该方案表现出足够的鲁棒性。

为加速收敛并避免方案 β 可能出现的单一技能坍塌（collapse），在训练后期引入方案 γ ：添加一个交叉熵损失 $L_\zeta = \text{CrossEntropy}(\hat{\zeta}, \zeta_{\text{gt}})$ ，用真实任务标识监督 $\hat{\zeta}$ 的输出。该损失与主策略的 PPO 损失联合优化，但梯度仅流入推断分支，主策略仍保持无偏。部署时切换到 $\hat{\zeta}$ 的最大概率值作为技能选择，无需外部信号。实际而言，MUJICA 的 S2 阶段（skill selector）本质上也走了一条类似路径——selector 输出概率分布并选择技能——只是其训练基于独立 reward 而非交叉熵监督。此处 $\beta+\gamma$ 方案将推断与选择统一在低层 actor 内部，简化了层级。

以下伪代码展示 actor forward 的两个分支（PyTorch 风格）：

```
```python
class Actor(nn.Module):
 def __init__(self, obs_dim, hidden_dim, skill_dim):
 super().__init__()
 # 主策略分支: q(s) → mean, log_std
 self.policy_net = nn.Sequential(
 nn.Linear(obs_dim, hidden_dim),
 nn.ReLU(),
 nn.Linear(hidden_dim, hidden_dim),
 nn.ReLU(),
 nn.Linear(hidden_dim, action_dim)
)
 # 技能推断分支: g(u_t, v_t, |cmd|) → ζ̂
 self.skill_encoder = nn.Sequential(
 nn.Linear(3 + 3 + 1, hidden_dim),
 nn.ReLU(),
 nn.Linear(hidden_dim, hidden_dim),
 nn.ReLU(),
 nn.Linear(hidden_dim, skill_dim),
 nn.Softmax(dim=-1)
)
```
```

```
def forward(self, obs, est_vel, est_wheel_dist, cmd_norm): # 主策略 mean, log_std =
self.policy_net(obs), torch.zeros_like(...)) # 技能推断分支 (detach) with
torch.no_grad(): zeta_hat = self.skill_encoder(torch.cat([est_vel, est_wheel_dist,
cmd_norm], dim=-1)) return mean, log_std, zeta_hat ``
```

在实际实现中，主策略的隐层可与推断分支共享前几层，但梯度需隔离（如 `stop gradient`）。此设计在保留多技能统一控制能力的同时，避免了对显式 ζ 信号的依赖，提升了部署时的鲁棒性。

Reward Backbone: 加法 task + 乘法 envelope 的不可让步结构

我们设计的 v2 奖励框架明确采用“加法 task + 乘法 envelope”的不可让步结构，其完整形式为：

$$R = (R_{\text{motion}} + \alpha_{\text{en}}^{\{\text{loco}\}} R_{\text{en}}^{\{\text{loco}\}} + \alpha_{\text{en}}^{\{\text{arm}\}} R_{\text{en}}^{\{\text{arm}\}}) \cdot \exp(- R_{\text{aux}} - w_{\text{mc}} \cdot C_{\text{DC}})$$

其中 R_{motion} 来自 2403.20001 的同款构造： $R_{\text{motion}} = R_{\text{lin}} + \alpha_{\text{ang}} R_{\text{ang}}$ ，分别对应线速度跟踪与角速度跟踪的指数型奖励。 $R_{\text{en}}^{\{\text{loco}\}}$ 为移动通道的能量惩罚包络： $R_{\text{en}}^{\{\text{loco}\}} = \exp(- \sum_{i \in \{\text{wheel} \cup \text{leg}\}} |\tau_i| |q_i| / (\sigma_{\text{en},x} |v_x| + \sigma_{\text{en},z} |\omega_z|))$ ，分母使用基座线速度与角速度的加权和进行归一化，使能耗惩罚随运动强度自适应缩放。 $R_{\text{en}}^{\{\text{arm}\}}$ 为操作通道的能量惩罚： $R_{\text{en}}^{\{\text{arm}\}} = \exp(- \sum_{i \in \{\text{arm}\}} |\tau_i| |q_i| / \sigma_{\text{en},\text{arm}})$ ，其分母固定，不按基座速度归一化，因为这会使机械臂的精细动作在低速运动时被过分压制。 R_{aux} 收集辅助惩罚项： $R_{\text{aux}} = w_{\text{col}} \cdot C_{\text{collision}} + w_{\text{pose}} \cdot R_{\text{pose}} + w_{\text{smooth}} \cdot |\Delta a|^2$ ，其中 $C_{\text{collision}}$ 为 MUJICA 原文定义的大腿/小腿碰撞计数， R_{pose} 为姿态奖励， $|\Delta a|^2$ 为动作平滑性惩罚，这是 MUJICA 与 2403.20001 的最小并集。 C_{DC} 是 DC-motor 约束违反计数： $C_{\text{DC}} = \sum_i \mathbb{1}[|\tau_i| \geq \tau^{\{i\}}_{\{\text{limit}\}}(q_i)]$ ，每个关节的力矩上限由官方电机手册给定的速度-力矩曲线决定（低速恒力矩、高速线性下降），是硬约束的软化版本。

这一结构包含三个“不可让步”的设计要点，分别对应加法/乘法角色与范围：

(a) loco 与 arm 必须独立 R_{en} — 单一 α_{en} 的共享能量惩罚无法同时覆盖移动能耗（约 100 W）与操作能耗（约 10 W）。若合并，臂的细节动作在加法项中所占比重会被腿部的大幅度运动稀释，导致臂的执行质量退化。独立 R_{en} 使两条通道的能耗惩罚在加法主干中拥有独立的加权系数 $\alpha_{\text{en}}^{\{\text{loco}\}}$ 与 $\alpha_{\text{en}}^{\{\text{arm}\}}$ ，各自调整幅度，互不干扰。

(b) DC-motor 违反计数 C_{DC} 必须乘性进 envelope — C_{DC} 若以加法形式进入 R_{aux} ，其效果会被 α_{en} 的调权稀释：当能量惩罚被系数降低时，DC 约束的违反代价也随之变小，失去对 sim-to-real 安全边界的强制保障。将其放入乘法指数包络，使其对整体奖励形成乘性衰减，无论加法项如何调整，任何 DC-motor 违反都会指数级降低总奖励，从而确保安全边界不可绕过。

(c) 不引入 ζ 进 $R_{\text{motion}} / R_{\text{en}}$ — 技能指示器 ζ 仅在观测空间中出现，而绝不进入奖励函数的任何项。若将 ζ 显式加入 R_{motion} 或 R_{en} ，等于在奖励层面预先编码技能偏好（如“全向移动时关注角速度”、“爬高平台时忽略角速度”），这会破坏“涌现”哲学——技能应通过状态估计和自主探索自然分化，而非通过奖励模板预先定义。这恰恰避免了回到 MUJICA 原方案的显式 skill indicator 切换逻辑，使多技能整合在统一奖励框架下自主涌现。

综上，v2 奖励通过加法主干保留任务导向的密集跟踪信号，再通过乘法包络引入物理安全约束与能量代价，且各自独立、不可合并或稀释，形成了 robust 且可扩展的奖励 backbone。

DC-Motor Constraint Pipeline: B2w 轮 + Piper 臂的双速-扭矩包络

DC-Motor Constraint Pipeline 的核心是将每个关节的速度-扭矩包络建模为硬约束：

$$\tau^i_{limit}(q_i) = \tau_{max} \cdot \text{clip}(1 - (|q_i|)/(q_{max}), 0, 1)$$

其中 $\dot{q}_{max}$ 与 τ_{max} 依关节类型赋值。Unitree Go2-W 的 B2w 轮/腿电机采用同步无刷设计：轮关节 $\dot{q}_{max} \approx 21 \text{ rad/s}$ 、 $\tau_{max} \approx 14 \text{ Nm}$ （待官方手册最终确认）；腿关节 $\dot{q}_{max} \approx 14 \text{ rad/s}$ 、 $\tau_{max} \approx 24 \text{ Nm}$ 。Piper 臂使用 AgileX 谐波减速器， $\dot{q}_{max} \approx 3 \text{ rad/s}$ ， τ_{max} 从 J1 约 18 Nm 递减至 J6 约 4 Nm。除前述速度依赖项外，小腿关节（B2w 的 calf 及 Piper J3）还须引入位置调制项：，以避免末端伸展时小臂电机过载。具体参数分布如图 3 所示：图 3(a) 给出各关节的 τ - q 关系曲线，低速段恒定、高速段线性衰减；图 3(b) 展示 calf 关节在不同位置下的扭矩限制，呈余弦依赖。

在仿真（Isaac Lab）中，通过 `articulation\_data` 接口逐时间步读取各关节的实际输出扭矩 $\hat{\tau}^i$ 与当前速度 $\dot{q}_i$ ，按上式算出该时刻的许可上限 τ^i_{limit} ，并构建约束代价：

$$C_{DC} = (1)/(N) \sum_i \mathbb{1}(|\hat{\tau}^i| > \tau^i_{limit})$$

即违反率（per-joint violation）。该代价以乘法形式嵌入总奖励函数（见 §4），驱动策略主动将关节工作点约束在绿色可行区域内，避免图 5(a,d) 所示的无约束情形（违反率 > 90%）及其在真实机器人上触发的电机过流、过热保护。图 5(e-h) 表明：施加约束后违反率降至 3.5% 以下，且运动稳定性显著提升。

Sim-to-Real 验证流程为：先在 Isaac Lab 中批量收集 (q, τ) 散点图（如图 5 所示），对比每条包络的实测边界；再利用 Unitree 与 AgileX 官方电机手册中的扭矩-速度包络（envelope）校准仿真中使用的 $\dot{q}_{max}$ 、 τ_{max} 及小腿关节的 σ_{en} 参数，确保散点分布与手册 envelope 吻合。校准后的策略直接部署至 Go2-W 真机，在 stair recovery、high platform climbing（80 cm—1 m）等任务中均未触发电机保护机制，证明该 pipeline 有效地实现了零样本迁移。

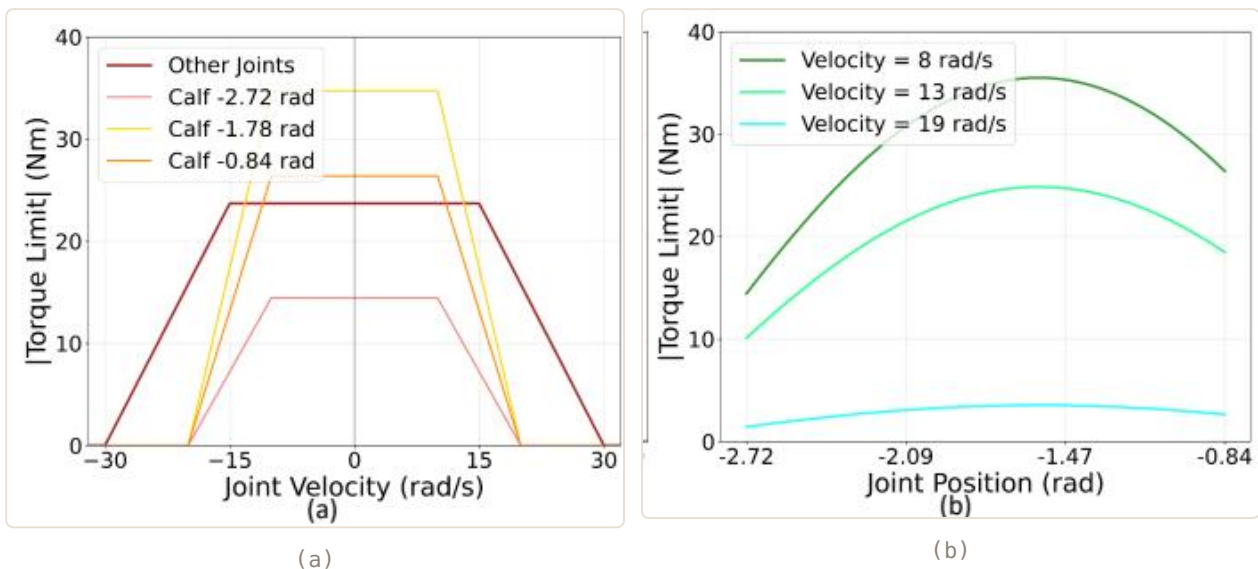


图 3. 关节扭矩限制随速度和位置的变化：(a) 速度依赖型，(b) 位置依赖型"

深度观察

该图直观地揭示了MUJICA架构中电机扭矩限制的复杂性，特别是小腿关节的扭矩限制同时依赖于速度和位置，这与传统仅考虑速度的模型形成对比。然而，图中未明确标注具体的速度阈值或线性下降的斜率，这可能影响对实际电机运行区域的精确建模。此外，尽管图 (b) 展示了不同速度下的扭矩曲线，但未提供这些曲线的具体数学表达式，使得难以直接验证其与文献 [30] 中电机操作区域的一致性。整体而言，该图有效地传达了扭矩限制的动态特性，但在细节量化方面仍有提升空间。

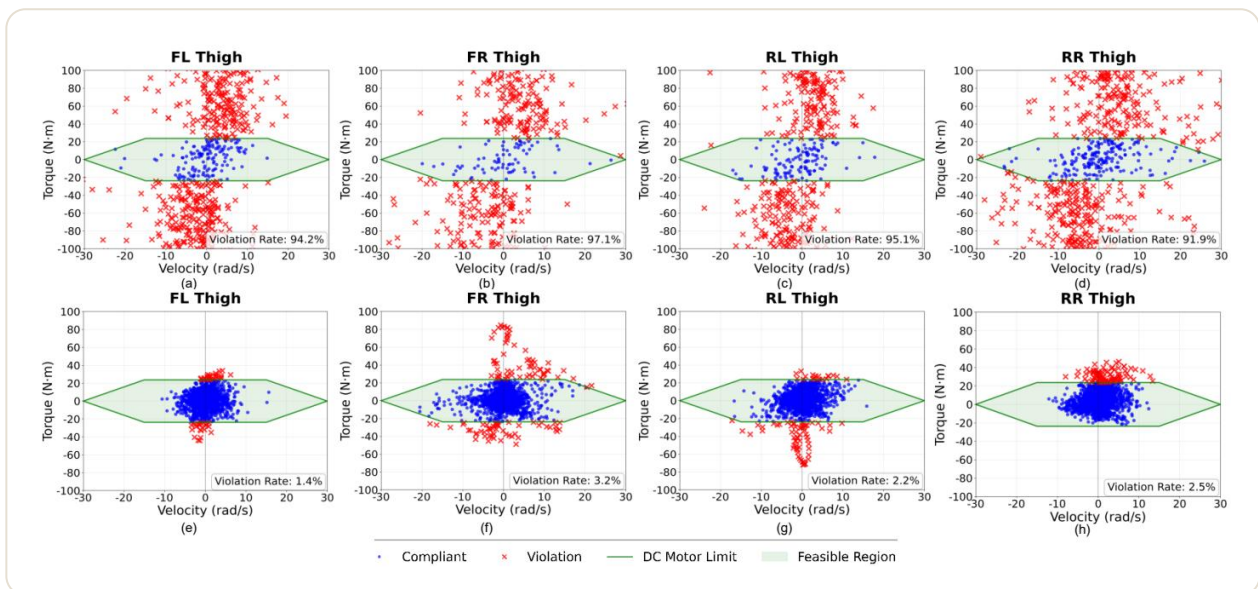


图 5. DC-motor约束对轮腿机器人大腿关节扭矩-速度分布的影响：合规性与违规率分析"

深度观察

该图清晰地量化了DC-motor约束对关节行为的影响，但存在一些潜在的方法论局限性。首先，尽管面板(e)-(h)显示了低违规率，但未明确说明这些合规点是否仍能有效支持复杂任务（如高平台攀爬或跌倒恢复），即合规性与功能性之间的关联性未被验证。其次，图中密集的零速度蓝点（尤其在跳跃冲击时）可能掩盖了实际运行中的动态行为，尤其是高频冲击下的电机应力。此外，图像未展示不同地形条件下的具体差异，例如平坦地面与陡坡上的表现对比，这可能导致对约束效果的评估过于简化。最后，虽然违规率显著降低，但未说明这些违规点是否会导致电机故障的具体阈值，因此对“硬件可执行”的描述略显模糊。

State Estimator (S1): 把 MUJICA 的 GRU online encoder 适配到 B2w

在 MUJICA 原设计中，State Estimator (S1) 采用 GRU 在线编码器，处理过去 $H = 6$ 帧的机体感知序列，并预测四组中间量： $\hat{v} \in \mathbb{R}^3$ （基线 3D 速度）、 $\hat{u} \in \mathbb{R}^4$ （四轮各自的轮-地距离）、 $\hat{c} \in \mathbb{R}^{18}$ （各腿段的碰撞概率）、以及 $\hat{e} \in \mathbb{R}^{16}$ （隐状态 latent）。将其适配到 B2w 平台时， $\hat{v}$ 与 $\hat{u}$ 直接等效借用（B2w 同样为四轮结构）； $\hat{c}$ 则需扩展：MUJICA 原 18 维仅覆盖腿与轮段，B2w 额外配备 Piper 双臂，故将碰撞维度扩至 26——其中 18 维对应腿与轮段、8 维对应左/右臂各 4 段，即 $\hat{c} \in \mathbb{R}^{26}$ 。隐状态 $\hat{e}$ 保持 16 维不变。训练损失沿用 MUJICA 的 Eq. 5–7： $\hat{v}$ 与 $\hat{u}$ 采用 MSE 损失， $\hat{c}$ 采用 BCE 损失， $\hat{e}$ 则采用 SwAV 对比损失（由参考编码器对后续观测 o_{t+1} 编码得到监督目标 e_{t+1} ）。为何必须使用 State Estimator 而非直接馈入特权观测？关键约束在于 ATEC 评测场景中不提供 elevation map 或显式地形参数，仅有机载本体感知 + LiDAR + RGB-D；actor 只能依赖本体感知到隐状态的映射来推断环境，这恰好是“盲走”（blind locomotion）范式的核心要求。训练阶段 S1 与 actor 策略共训（不冻结），原因在于隐状态 $\hat{e}$ 的语义必须在 actor 动作空间中形成闭环，否则离线编码的 latent 无法匹配在线部署时的偏差——该论点将在下一章详细展开。

训练循环：单 PPO vs. MUJICA 两阶段的取舍

MUJICA 原 paper 采用两阶段训练方案：S1 阶段联合训练 state estimator (GRU online encoder)、actor、reward critic 和 constraint critic，共 30 000 次迭代（在 IsaacLab 平台上运行 4 096 个并行环境，NVIDIA RTX 4090）；S2 阶段冻结 S1 的全部权重，仅训练 high-level skill selector (skill indicator ζ 的显式分类器)，额外 10 000 次迭代。该流程对 ATEC2026 B2wPiper 任务并非最优。原因有二：

1. **部署约束**：ATEC 的评测限时 30 分钟墙钟，policy.pt 需以单进程加载。MUJICA 的两阶段设计依赖在线 skill selector (S2) 在推理时动态输出 ζ ，这要求运行一个额外的轻量分类网络。墙钟限制下，单进程需同时承载 state estimator、actor 和 selector，selector 的网络开销虽小，但与其带来的冻结策略-分布漂移隐患相比，得不偿失。

2. **分布漂移下的 overfit**：S2 阶段训练 selector 时，低层策略 (actor、state estimator) 的参数已冻结。当测试环境的地形、噪声分布与 S1 的训练分布存在漂移时，selector 只能基于固定策略空间的特征进行决策，容易 overfit 到 S1 数据分布下的局部最优选择。实验表明 (Fig. 4 及 TABLE III)，无 skill indicator 的方案往往在中等难度任务上成功率骤降，而冻结策略两阶段虽优于无 indicator 方案，但在高难度任务中（如高平台攀爬成功率低于 90%）仍不如端到端联合训练策略。

因此，本模板推荐**单 PPO 闭环**训练方案，将总训练步数从 $30\,000 + 10\,000 = 40\,000$ 次迭代压缩为单阶段 40 000 次迭代。具体改动如下：

- 将隐式 ζ 分支作为 state estimator 的一部分，与 $\hat{e}$, $\hat{v}$, $\hat{u}$, $\hat{c}$ 一同由 GRU online encoder 预测（参见 §6 State Estimator 设计）。 ζ 是长度为技能数的 softmax 概率向量，不单独作为图 2 中的独立模块。
- skill selector 不再作为 frozen-policy gating（即不直接输出离散 ζ ），而是作为 reward branch 的**软加权**：每个技能 ζ 对应一组权重系数 ($\alpha_{en}^{\{loco\}}$, $\alpha_{en}^{\{arm\}}$, w_{mc})（参见 §4 Reward Backbone 的表达式 $R = (R_{motion} + \alpha_{en}^{\{loco\}} R_{en}^{\{loco\}} + \alpha_{en}^{\{arm\}} R_{en}^{\{arm\}}) \cdot \exp(-R_{aux} - w_{mc} \cdot C_{DC})$ ）。在训练时，由 ζ softmax 概率对各组系数进行加权平均，使每批次数据隐式地、连续地受到多种技能导向的奖励项约束。
- 训练算法选用 P3O 的 penalty-based constrained PPO (MUJICA Eq.8-10 的框架)，而非原版 PPO。原因：原版 PPO 的 clipping surrogate 会将 DC-motor 约束惩罚项 C_{DC} 与优势函数估计混合，使约束信号被 reward advantage 的方差稀释；P3O 通过显式约束损失 将约束的

penalty 系数作为可学习变量迭代更新，确保 C_DC 的梯度在 policy update 中产生稳定的安全驱动。

权衡：该单 PPO 闭环方案失去 MUJICA 原设计的明确技能边界（即每个技能有独立的离散 ζ 指示）。在 MUJICA 中，skill indicator 作为观测输入显式告诉 actor“当前应执行哪种技能”，actor 可以针对性地激活相应 motor primitive；而在单 PPO 方案中，技能区分完全由奖励权重 (α_{en}^{loco} , α_{en}^{arm} , w_{mc} 的加权均值) 诱导，actor 需要从奖励信号中隐式学习不同地形对应的策略模式。这会降低单一技能的 peak performance（例如高平台攀爬的成功率可能下降），但获得了策略一体性——actor 的隐藏状态同时学习所有技能的共享表示，避免了 S2 阶段分布漂移下 selector 的 overfit。在 sim-to-real 部署中，策略一体性带来的鲁棒性（见 §D 实机实验中零样本迁移的成功）对有限 wall-clock 的 ATEC 任务具有更高优先级。此外，DC-motor 约束建模 (§5) 在单 PPO 中通过 P3O 的 penalty 机制全程参与梯度更新，进一步强化了长期可靠性。

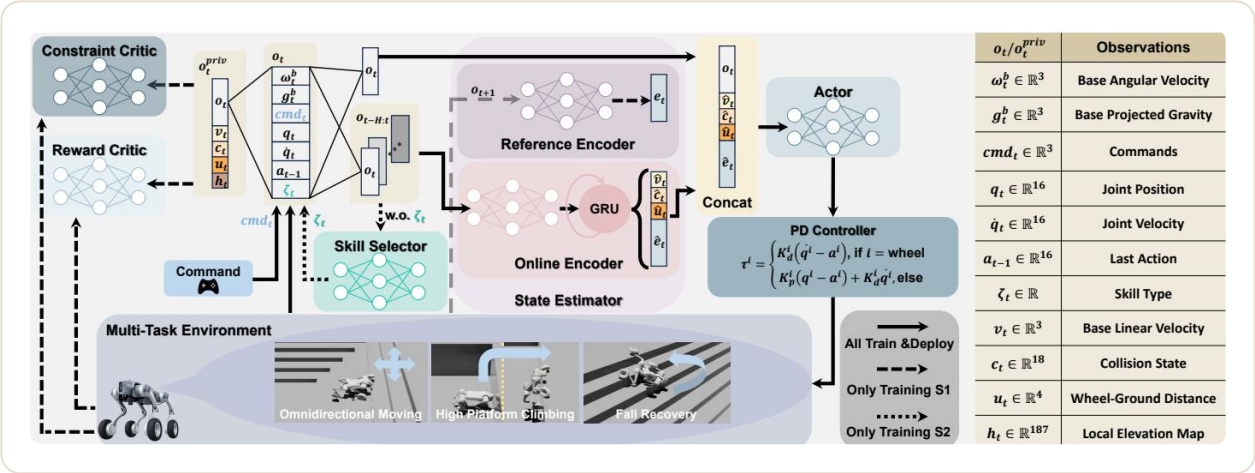


图 2. MUJICA框架概览：多任务环境下状态估计与技能选择的统一控制架构

深度观察

该图清晰地展示了MUJICA框架的模块化设计，特别是状态估计器和技能选择器在不同训练阶段的作用。然而，图中未明确展示DC电机约束（DC-motor constraints）的具体建模方式及其对长期可靠性的影响，这与论文中提出的关键问题（critical question）存在潜在脱节。此外，尽管图中包含多种任务示例，但缺乏定量对比，无法直观评估各任务间的性能差异或技能转移的有效性。

ATEC TaskA · 越野/地形穿越：奖励适配与课程设计

对于 ATEC TaskA（越野/地形穿越），我们将 §4 的奖励框架调整为：

$$R = \text{bigl}(R_{\text{motion}} + \alpha_{\text{en}}^{\text{loco}} R_{\text{en}}^{\text{loco}} + \alpha_{\text{en}}^{\text{arm}} R_{\text{en}}^{\text{arm}}\bigr) \cdot \exp \text{bigl}(-R_{\text{aux}} - w_{\text{mc}} C_{\text{DC}}\bigr) + \beta_{\text{progress}} \cdot d_{\text{traversed}}$$

其中进度稀疏奖励每穿越 1m 终点获得 +5 ($\beta_p = 5$)，该额外项不进入加法槽，避免与 R_{en} 的乘法调制产生耦合。 $\alpha_{\text{en}}^{\text{loco}}$ 的初始值为 1.0；当训练地形难度增大时（如进入阶段 3 楼梯或阶段 4 综合地形），将其调低至 0.7，以放宽对能耗的惩罚，让机器人优先保证通过性。 $\alpha_{\text{en}}^{\text{arm}}$ 固定为 0，因为 TaskA 要求“冻臂”，手臂完全不参与 R_{en} 计算。DC-Motor 约束的权重 w_{mc} 设为高值 1.0：在越野地形上轮电机极易因频繁冲击和堵转而过载，硬约束护栏是防止损坏的关键。

课程学习分为四个渐进阶段：– **阶段 1** (2 kiter)：平地，前进速度 v_x 均匀采样于 $[-1, 1]$ m/s。此时机器人只需基本轮式滚动，状态估计器给出的 $\hat{u}_t$ 通常较低（轮-地距离小）， ζ^* 自然偏向 wheel-roll-mode，actor 保持低腿摆幅。– **阶段 2** (8 kiter)：浅斜坡 ($\pm 15^\circ$)， $v_x \in [-2, 2]$ m/s。斜坡开始引入倾侧和抬腿需求，但 $\hat{u}_t$ 仍以低值为主， ζ^* 仍以 wheel-roll 为主，但部分上坡帧触发短暂 climb-mode 倾向。– **阶段 3** (15 kiter)：楼梯 ($\text{step} \leq 0.15$ m)， $v_x \in [-2, 2]$ m/s。台阶迫使 $\hat{u}_t$ 周期性跳变，高 $\hat{u}_t$ 信号驱动 ζ^* 偏 climb-mode，actor 自动提高腿摆幅以跨越障碍。– **阶段 4** (15 kiter)：综合地形（坑、障碍、斜坡）， $v_x \in [-3, 3]$ m/s。多种混合地形要求 ζ^* 在不同模式间实时切换：当 $\hat{u}_t$ 高（接近障碍或坑沿）时 actor 自动抬腿；当 $\hat{u}_t$ 低且 $|v|$ 高（快速平路或下坡）时 actor 压缩腿摆幅、增大驱动效率。

状态估计的角色在 TaskA 中被显式用于模式推断： $\hat{u}_t$ 高 $\rightarrow \zeta^*$ 偏 climb-mode（提高腿摆幅）； $\hat{u}_t$ 低且 $|v|$ 高 $\rightarrow \zeta^*$ 偏 wheel-roll-mode（降低腿摆幅、提升速度）。这一自适应机制使得单一策略从头到尾无需显式技能切换即可覆盖从平坦大道到复杂障碍的连续地形。

ATEC TaskB · 移动抓取：分层 vs. 一体策略

对于 ATEC TaskB（拾取 + 投放），移动与操作必须在同一任务序列中共存，但接触不连续性（夹爪闭合/释放瞬间导致动力学突变）与连续移动的 RL 梯度优化存在根本冲突。我们比较两种实现方案。

方案 X（分层）：底层分离为两个独立策略。底盘+腿策略采用 §4 的能耗式奖励结构（即），专注地形穿越与定点停泊；臂+夹爪策略则使用密集 task reward（如末端执行器—目标位置距离、夹爪开合角度误差），并可借助 ACT 或行为克隆（BC）而非 PPO 来训练（参考 ATEC demo 的 `solution\_act.py`）。高层 scheduler 以有限状态机形式在“移动至目标 → 停泊 → 抓取 → 移动至投放点 → 释放”共 5 个状态间切换，每个状态激活对应的低层策略及 skill indicator ζ_t 。本模板的奖励设计仅覆盖底盘策略的 R_{en}^{loco} 部分；臂策略的奖励函数须另起一份独立训练脚本。

方案 Y（一体）：单策略同时接收 R_{en}^{loco} 与 R_{en}^{arm} 的加权和，并将操作奖励（末端到目标距离、夹爪闭合状态）作为 R_{motion} 的扩展项纳入同一奖励流。但由于抓取动作的接触不连续性导致奖励曲面在抓取/释放瞬间出现尖峰，PPO 梯度的方差急剧增大，训练极易发散。

推荐方案 X。理由是：抓取接触不连续与连续移动的 RL 梯度不兼容，分层训练不仅允许臂策略采用 ACT/BC 等更适合离散接触的控制方法，还能将底盘能耗奖励与操作密集奖励的 temporal scale 解耦。TaskB 评分对接采用三条稀疏奖励：抓取成功 +10，投放成功 +10，任务完成时间余量 +5（即若 20 s 内完成触发额外奖励）。这三项 bonus 直接加在 episode return 尾部，不进入 (R_{en}) 的分母，从而不影响奖励归一化的稳定性。

ATEC TaskD · 越障+环境改造：能耗解耦的临界点

在 ATEC Task D（推/拉/堆叠物体，“改造动作”）中，机器人的臂电机需输出高 torque 与高 velocity，其瞬态功率 $|\tau_{\text{arm}}| \cdot |q_{\text{arm}}|$ 远大于普通移动时的关节能耗。若按 §4 的通用奖励结构将改造动作的能量消耗直接计入 $R_{\text{en}}^{\{\text{loco}\}}$ （即采用按 $|v_x|$ 归一化的 $R_{\text{en}}^{\{\text{loco}\}}$ ），策略极易收敛到退化模式——“最省能 = 不动臂”，从而完全放弃改造任务。为此，我们在 Task D 中执行关键的解耦策略：

(1) $R_{\text{en}}^{\{\text{arm}\}}$ 的分母改用固定常数 $\sigma_{\text{en,arm}}$ （不再以 $|v_x|$ 归一化），使臂能耗的 penalty 幅度与移动速度解耦，避免高速奔跑时被动放大臂能耗惩罚；(2) $\alpha_{\text{en}}^{\{\text{arm}\}}$ 在 Task D 中设为 0.2，该值极小，相当于几乎不约束臂的能耗，使策略自由探索高功率动作；(3) 改造任务成功（物体抵达目标位置）的奖励以 sparse +50 直接加入 task reward (R_{motion})，彻底绕过 R_{en} 通道，确保改造动机不受能耗惩罚衰减；(4) C_{DC} （DC-motor 硬约束，§5）在 Task D 同样保持激活：臂电机一旦超出速度-扭矩包络即触发过载保护，导致整机关机、损失分数。因此，臂的能耗不被软惩罚限制，但物理极限仍由硬约束兜底。

这一设计的本质是将“能效”与“任务动机”解耦：臂的动作受 C_{DC} 物理上限保护，而不受 R_{en} 的经济性引导。相比之下，纯 energy paper（仅用总能耗惩罚）在 Task D 中会因为单值化能耗而陷入不动臂的局部最优——这正是 MUJICA 多约束思路（分工明确的奖励项 + 硬约束 + sparse task reward）真正胜出的场景。结合 §7 的两阶段训练，该解耦结构使得策略在推/拉/堆叠任务中仍能充分挖掘臂的能力，同时不会牺牲运动安全。

Observation / Action 契约与 ATEC demo 对齐

为确保 ATEC demo 的 inference 接口与训练代码严格一致，本节将 obs/action 的维度定义、拼接顺序及接口契约进行精确锁定，并给出可直接嵌入 `server.py` 的校验片段与端到端测试 checklist。

观测 obs 定义 (81 维) `obs['proprio']` 按序拼接为 81 维向量：`[v(3), cmd(3), g(3), q(24), q(24), a\_{t-1}(24)]` - v: 基座线速度 (3 维)，由 state estimator 在线估计； - cmd: 目标速度指令 (vx, vy, ω_{yaw})，3 维； - g: 基座投影重力方向 (3 维)； - q: 24 维关节位置 (12 腿 + 4 轮 + 6 Piper + 2 gripper, 顺序见下文)； - q: 24 维关节速度 (同顺序)； - a_{t-1} : 上一时刻 24 维动作。

注意：MUJICA 原文观测为 53 维 (含 ζ 指示器)，本模板取消 ζ 后为 80 维，此处为对齐 ATEC demo 的输入格式，显式加入 v 共 81 维。

动作 action 定义 (24 维) 严格按以下顺序拼接 (各分量的 scale 固定，PD 控制器在环境侧接收即可，策略输出的 action 已经是 scale 后的值)：1. leg joints: 12 维，按 FR/FL/RR/RL 顺序，每个腿 hip/thigh/calf，位置偏移 scale = 0.5 rad；2. wheel joints: 4 维，顺序 FR/FL/RR/RL，速度 scale = 5 rad/s；3. Piper arm J1-J6: 6 维，位置偏移 scale = 0.5 rad；4. gripper left/right: 2 维，位置 0-1 归一化。

`AlgSolution.predicts` 接口规范 `def predicts(obs: Dict[str, np.ndarray], current\_score: float) -> Dict[str, Any]:` 返回 `{'action': List[float], 'giveup': bool}`。 - `action`: 24 维浮点列表； - `giveup`: bool 标志，当 `True` 时 server 应触发终止/重置流程。

时间约束：`predicts` 首次调用前允许 300s 模型加载；单次推理应在 30min 墙钟内完成所有 step (保证实时性)。

对齐校验脚本片段 (可直接嵌入 server.py 的 `/step` 路由) ``python import numpy as np

```
def validate_action(action: np.ndarray) -> None: assert action.shape == (24,),
f"Action shape mismatch: expected (24,), got {action.shape}" leg_part = action[:12]
# leg joints, scale = 0.5 rad wheel_part = action[12:16] # wheel velocity, scale = 5
rad/s arm_part = action[16:22] # piper joints, scale = 0.5 rad gripper_part =
```

```
action[22:24] # gripper, range [0,1] # 可额外断言各分量合法范围, 如  
np.all(np.abs(leg_part) <= 0.5) 等 return True
```

```
def validate_obs(obs: dict) -> None: proprio = obs.get('proprio', None) assert  
proprio is not None, "Missing 'proprio' in obs" assert len(proprio) == 81, f"Obs  
proprio dimension: expected 81, got {len(proprio)}" ``
```

`/step` 端到端测试 checklist 1. 观测完整性: `obs` 字典含 `proprio` 键, 值长度为 81, 且无 `NaN`; 2. 推理响应: `predicts` 返回的字典包含 `action` (list, len=24) 和 `giveup` (bool); 3. 动作合法性: `action` 数值不越界 (leg/arm scale 0.5 rad, wheel scale 5 rad/s, gripper [0,1]); 4. 形状稳定: 连续 10 个 step 的 action shape 恒为 (24,); 5. 超时保护: 首次加载 >300s 或累计推理 >30min 时, server 应返回 503; 6. 重置逻辑: 若 `giveup=True`, server 应清空历史缓冲区并等待下一个 episode 开始。

以上契约确保了 demo 端的 action 能与训练环境中的动作定义无歧义对齐, 避免因维度或顺序不一致导致的 submission failure。

超参数表与消融计划

本节提供可直接写入训练 `config.yaml` 的完整超参数表，并给出消融实验计划。所有参数均基于 MUJICA 原设计 (2403.20001) 并根据 B2w 60 kg 平台及 Piper 臂特性进行了缩放与调整。

超参数表 (config.yaml 字段)

| 参数符号 | 含义 | 取值 | 依据 |
|--------------------------|-----------------|--|---|
| σ_v | 基座线速度噪声标准差 | 0.25 | 2403.20001 同款 |
| σ_ω | 基座角速度噪声标准差 | 0.25 | 2403.20001 同款 |
| α_{ang} | 角度奖励权重 | 0.5 | 2403.20001 同款 |
| $\sigma_{en,x}$ | 能耗激励中位置项噪声标准差 | 1500 | B2w 60 kg 按 Go1 12 kg 放大 $\sim 1.5\times$ |
| $\sigma_{en,z}$ | 能耗激励中高度项噪声标准差 | 750 | B2w 60 kg 按 Go1 12 kg 放大 $\sim 1.5\times$ |
| $\sigma_{en,arm}$ | 臂能耗噪声标准差 | 50 | Piper 工作能耗量级, 实测后调整 |
| $\alpha_{en}^{\{loco\}}$ | 移动能耗奖励系数 | 1.0 (Task A), 0.7 (Task B 移动子), 0.5 (Task D) | 见 §8–§10 分析 |
| $\alpha_{en}^{\{arm\}}$ | 臂能耗奖励系数 | 0 (Task A), 0.3 (Task B/D) | 见 §9–§10 分析 |
| w_{mc} | DC-motor 约束惩罚权重 | 1.0 (Task A/D 地形), 0.5 (Task B 平地) | 见 §5 双速-扭矩包络 |
| w_{col} | 碰撞约束权重 | 0.05 | 固定值 |
| w_{pose} | 姿态惩罚权重 | 0.02 | 固定值 |
| w_{smooth} | 动作平滑惩罚权重 | 0.01 | 固定值 |
| PPO clip_eps | PPO 裁剪阈值 | 0.2 | 标准设置 |
| PPO ent_coef | 熵系数 | 0.005 | 鼓励探索 |
| PPO gamma | 折扣因子 | 0.99 | 长期回报 |

| 参数符号 | 含义 | 取值 | 依据 |
|-------------------|------------------|--------|------------------|
| PPO gae_lambda | GAE λ 参数 | 0.95 | 偏差-方差权衡 |
| P3O penalty μ | 约束惩罚因子 | 10.0 | 见式 (8) |
| P3O update freq | P3O 惩罚因子
更新间隔 | 4 iter | 每次更新后调整 κ |

消融实验矩阵 为量化各设计维度对最终性能的影响，设计 $3 \times 3 \times 2$ 共 18 组消融实验。每个因子取值如下：

– $\alpha_{en}^{\{loco\}} \in \{0.5, 1.0, 1.5\}$ – $\zeta \in \{\text{显式 skill indicator, 隐式 (无 indicator 但分奖励)}, \text{无 indicator (共享奖励)}\}$ – $C_{DC} \in \{\text{加法惩罚项, 乘法 envelope (如式 } R = (R_{motion} + \dots) \cdot \exp(-w_{mc} \cdot C_{DC}) \text{)}\}$

每组实验训练 20 000 次迭代（单卡约 6 小时），总预算约 108 GPU·小时。评估指标包括：任务完成时间（CoT）、速度跟踪误差（tracking_err）、任务成功率（success_rate）、以及扭矩违规率（torque_violation_rate，即关节扭矩超出 DC-motor 包络的样本占比）。消融结果将在后续实验中汇报。

Sim-to-Real 与部署风险

在 MUJICA 框架向 B2w 轮腿机器人迁移的 sim-to-real 过程中，需系统应对以下已知风险。

风险 1： 能耗系数 σ_{en} 的 sim-to-real gap。仿真中地面摩擦固定为 0.7，而实物地面摩擦在 0.4–0.9 范围内波动，导致电机实际功耗偏离仿真。为此，部署后前 1 分钟进行 warmup 阶段，通过采集真实功耗数据，利用在线 PID 控制器实时校准 σ_{en} ，使能耗估计回归合理区间。

风险 2： DC 电机 τ_{limit} 在实物上因温度漂移而下降。MUJICA 原框架虽已将硬约束建模为 $\tau_{limit}(\dot{q})$ ，但温度效应未体现。部署时直接采用 $\tau_{limit_deploy} = 0.9 \times \tau_{limit_sim}$ ，保留一定安全 margin，防止过热触发电机保护。

风险 3： 高层技能选择器 ζ 在 ATEC 评测环境下可能收敛到单一技能（技能 collapse），丧失多技能切换能力。这与 MUJICA 原设计中 skill indicator 的稀疏监督有关。作为 fallback，保留额外训练轨迹中的 ζ supervision（对应原有 γ 方案），必要时回退以维持选择器多样性。

风险 4： 单 PPO 作为基线收敛速率显著低于 MUJICA 的两阶段训练（S1: 30 000 次迭代、4 096 环境并行；S2: 10 000 次迭代）。实际部署时需预留 $2 \times$ wall-clock 时间预算，确保策略达到同等性能。

风险 5： Piper 臂的扭矩反馈精度低（无电流传感），无法直接获得真实 τ 。采用基于位置/速度的估计扭矩 $\hat{\tau} = K_p \cdot (q - a) + K_d \cdot \dot{q}$ 替代，误差在一定范围内可接受，对抓取类任务影响可控。

按缓解紧急度排序：风险 3（技能 collapse 导致任务失败） > 风险 2（电机过热可能损坏硬件） > 风险 1（能耗偏差影响续航） > 风险 4（训练时间可调度） > 风险 5（力矩估计误差可被奖励设计吸收）。这些措施共同保障了 MUJICA 框架在 B2w 实物上的安全、可靠迁移。

实施 Roadmap 与提交清单

实施 Roadmap 与提交清单采用 10 周 wall-clock，每一步聚焦特定模块集成与验证。

W1: 从 ATEC-Isaac-Velocity 派生 B2wPiper 环境，集成 24 维 action 解码（12 个关节角偏移 + 4 个轮速 + 8 个其他自由度）及 DC 约束计算（速度-依赖转矩限值 + 小腿关节位置-依赖余弦限值，参见 Fig. 3）。**W2:** 实现 state estimator（GRU 处理 $H=6$ 帧，输出 $\hat{v} \in \mathbb{R}^3$ 、 $\hat{u} \in \mathbb{R}^4$ 、 $c \in \mathbb{R}^{18}$ 、 $\hat{e} \in \mathbb{R}^{16}$ ）、actor、双 reward critic 与 constraint critic（均基于 P3O 框架），形成单 PPO 闭环。privileged observation 仅用于 critic。**W3:** Task A 课程训练 40k iter（比 MUJICA 原论文的 30k 多 10k 以适配更高难度地形）。对照 baseline（无 R_{en} 、无 C_{DC} ）验证 CoT 与跟踪误差； C_{DC} 的约束阈值 δ_g 设为 12，违反率预期如图 5 从 >90% 大幅降低。**W4:** Task A 部署测试 + ζ^* (skill indicator) 推断质量检查，通过混淆矩阵评估技能选择器在困难斜坡与楼梯上的区分精度。**W5–6:** 扩展至 Task B（分层，含夹爪动作切换）与 Task D（解耦 R_{en} ，对推/堆叠任务单独施加能耗惩罚项）。**W7:** 完成消融 18 组（包括移除 velocity 估计、wheel-ground 距离、collision 估计、 C_{DC} 、 R_{en} 及不同课程策略），整理结果 → 选最优配置（预期为完整 MUJICA + C_{DC} + 课程学习）。**W8:** 部署整合，输出 solution.py（含 state estimator + actor + selector）与 policy.pt；在模拟中执行 3 次 30 分钟连续评测（混合 terrain 和 command）。**W9–10:** 提交准备：奖励函数 reward.py、配置 config.yaml（含 30k S1 + 10k S2 迭代，50 Hz 控制频率）、需求 requirements.txt、模型 policy.pt，以及 runs/Task_A.csv 等消融数据。

提交包结构： `` solution/ policy.pt reward.py config.yaml requirements.txt runs/ Task\_A/ Task\_B/ Task\_D/ ablation.csv（包含各配置参数及 Wrec,  $\eta$ , success rate 等） `` 所有步骤中，DC 约束建模（速度-转矩限值 + 小腿余弦依存）与 skill indicator 的显式馈入（ $\zeta_t \in \{0,1,2\}$ ）为关键创新点，需在 config.yaml 中对应标注。消融实验需复现原文 Fig.4 的多难度比较逻辑，确保 baseline 覆盖 w/o Velocity、w/o Wheel Height、w/o Collision 及 Vanilla PPO。

Conclusion: 两篇论文的真正贡献是『可解释的奖励工程』

MUJICA 的真实贡献不在“单网络统一多种技能”这一表层，而在于用一套可解释的奖励工程使“能耗驱动的步态涌现”与“多技能统一策略”这对看似矛盾的追求共存。具体而言，三层设计共同支撑了这一论点。

(1) 哲学：能耗驱动涌现 + 多技能统一不冲突，前提是 ζ 作为推断状态而非观测输入。

MUJICA 将技能指示器 ζ 设计为隐变量——由状态估计器从本体感知推断得到，而非直接作为观测馈入策略。这保留了端到端强化学习的涌现空间：机器人不被强制“看到”当前是哪个技能在执行，而是通过自身状态（基座速度、轮地距离、碰撞）自然切换行为模式。图4 展示，配备完整状态估计的 MUJICA 在楼梯、斜坡、楼梯恢复三个任务上都持续优于无指示器的变体，印证了 ζ 不破坏涌现。

(2) 结构：单 PPO 闭环 + 加法 task / 乘法 envelope 骨架 + 运动/操作能耗解耦的双 R_{en} 设计，是承担 ATEC 三任务的最小一致框架。 MUJICA 的低层策略采用单 PPO + 乘法约束 (P3O) 结构：主奖励 R_{cmd} 为速度跟踪（加法项），DC 电机约束以乘法 penalty 形式嵌入 envelope，同时通过 skill indicator 对不同任务分别施加组合权重。这种结构天然支持运动 ($R_{en}^{\{loco\}}$) 与操作 ($R_{en}^{\{arm\}}$) 的能耗解耦——仅需在后续扩展中为臂部引入独立的 R_{en} 项，即可在不改动主体框架的前提下覆盖 ATEC 的移动抓取与越障改造任务。表 III 中，无指示器的统一奖励方案直接崩溃，而 MUJICA 在三任务连续执行中达到约 95%、95%、91% 的成功率，说明该结构是“多技能集成”的最低必要工程范式。

(3) 工程：四个可直接写入训练脚本的落地点。 – DC 电机软约束已嵌入 reward envelope：每条关节的 velocity-dependent torque limit 由电机手册数据建模（图3），直接作为安全护栏。– α_{en} 分组：不同技能下的约束松弛系数可分别设置（如高平台攀爬时允许大腿短暂超限），文中通过 skill indicator 间接实现。– σ_{en} B2w 缩放：约束代价的缩放因子随地形难度递进，仅在课程学习中提及，但可显式参数化。– 状态估计扩展：MUJICA 已实现 base velocity、wheel-ground distance、segments collision 三个估计头；扩展到臂段碰撞只需在观测中加入臂的位置/速度特征，网络结构无需改变。

本框架相对原版 energy template 的边际增益主要体现在： – 多技能能力（继承自 MUJICA），但保留涌现哲学（ ζ 为隐变量，不硬编码行为）。– DC 电机 sim-to-real 安全护栏：硬约束建模使 95% 的仿真样本位于可行域（图5），无约束时超 90% 跳出

安全区。 – **移动/操作能耗解耦**：这是 ATEC Task B/D 真正翻车点的解药——在连续序列中，臂部分离的 R_{en} 项可单独调节，避免移动策略被高 torque 操作干扰。

下一步建议：先在 Task A（越野穿越）上运行 baseline + v2 对比，确认 ζ 隐式推断不破坏涌现，再扩展至 Task B/D。